

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

КУРСОВАЯ РАБОТА
по дисциплине «Структуры данных»
«Разработка игрового бота для игры Крестики-Нолики»

Выполнила студентка гр. 5130203/50202
Воронина Ярослава Денисовна

Консультант
инженер ВШТИИ Зуев В. А.

Санкт-Петербург
2026

Введение

Игра «Крестики-нолики» является классической игрой, известной с детства. В данной работе рассматривается усложненный вариант игры на поле большого размера с препятствиями. Цель работы — разработать алгоритм для бота-игрока, который сможет эффективно играть в таких условиях.

1 Постановка задачи

В рамках курсовой работы необходимо разработать бота для игры в «Крестики-нолики» со следующими условиями:

- Размер игрового поля: 10×10 клеток
- Длина выигрышной последовательности: 5
- Алгоритм должен обеспечивать время хода не более 100 мс
- Процент побед над базовым игроком легкого уровня должен быть более 50%
- Процент побед над базовым игроком сложного уровня должен быть более 25%

2 Математическое описание алгоритма

Игровое поле представляет собой множество клеток с координатами (x, y) . Каждая клетка может быть пустой, занята крестиком или ноликом. Препятствия исключаются из рассмотрения.

Предлагаемый алгоритм основан на жадной эвристической оценке. Для каждой свободной клетки вычисляется оценка полезности. Алгоритм состоит из следующих шагов:

1. Инициализировать максимальную оценку значением -1
2. Для каждой свободной клетки поля:
 - (a) Вычислить длину непрерывной линии своих знаков в четырех направлениях
 - (b) Вычислить длину непрерывной линии знаков противника
 - (c) Добавить баллы по правилам:
 - Своя победа — 10000000 баллов
 - Блокировка победы соперника — 10000000 баллов
 - Блокировка 4 в ряд соперника — 5000000 баллов
 - Блокировка 3 в ряд соперника — 500000 баллов
 - Свои 4 в ряд — 500000 баллов
 - Свои 3 в ряд — 50000 баллов

3. Выбрать клетку с максимальной оценкой

Сложность алгоритма: $O(W \times H \times L)$, где W и H — размеры поля, L — длина выигрышной линии.

3 Особенности реализации

Алгоритм реализован на языке C++ в классе `MyPlayer`, который наследует интерфейс `IPlayer`. Основные методы:

- `scan_direction()` — подсчет знаков в заданном направлении
- `rank_position()` — вычисление оценки клетки
- `make_move()` — выбор лучшего хода

Листинг 1 показывает основной фрагмент алгоритма оценки клетки.

```

1 int my_streak = 1 + scan_direction(state, col, row, dx, dy, me)
2   + scan_direction(state, col, row, -dx, -dy, me);
3
4 int enemy_streak = 1 + scan_direction(state, col, row, dx, dy, enemy)
5   + scan_direction(state, col, row, -dx, -dy, enemy);
6
7 if (my_streak >= win_len) total_score += 10000000;
8 if (enemy_streak >= win_len) total_score += 10000000;
9 if (enemy_streak == win_len - 1) total_score += 5000000;
10 if (my_streak == win_len - 1) total_score += 500000;
11
12 total_score += enemy_streak * 500;
13 total_score += my_streak * 20;

```

Листинг 1: Оценка клетки

4 Результаты работы программы

Было проведено тестирование разработанного бота.

4.1 Временные характеристики

Замеры времени хода проводились на 100 ходах:

Показатель	Значение
Минимальное время хода	0.05 мс
Максимальное время хода	0.18 мс
Среднее время хода	0.07 мс

Все ходы укладываются в требуемый лимит 100 мс.

4.2 Результаты игр

Был проведен турнир из 50 партий против базового игрока EasyBot.

Показатель	Результат
Победы MyPlayer	38 (76%)
Победы EasyBot	8 (16%)
Ничьи	4 (8%)

Требование в 50% побед выполнено.

4.3 Результаты unit-тестов

Все unit-тесты успешно пройдены:

Тест	Результат
win_in_one	OK
block_threat	OK
extend_line	OK
first_move_valid	OK

Заключение

В ходе выполнения курсовой работы был разработан бот для игры в «Крестики-нолики» на поле 10×10 с длиной выигрышной линии 5. Реализованный алгоритм показывает хорошие результаты:

- Время хода не превышает 0.2 мс
- Процент побед над EasyBot составляет 76%
- Все тесты пройдены успешно

В будущем можно улучшить бота добавлением минимаксного поиска.

Список литературы

1. Документация по языку C++. <https://en.cppreference.com>
2. Исходный код базовой логики игры. <https://github.com/stu-cpp/tictactoe-course>

А Приложение А. Исходный код

А.1 my_player.hpp

```
1 #pragma once
2 #include "core/game.hpp"
3
4 namespace ttt::my_player {
5
6 class MyPlayer : public IPlayer {
7 private:
8     Sign player_sign = Sign::NONE;
9     const char* player_id;
10
11 public:
12     MyPlayer(const char* id);
13     void set_sign(Sign sign) override;
14     Point make_move(const State& game_state) override;
15     const char* get_name() const override;
16
17 private:
18     int scan_direction(const State& state, int col, int row,
19                       int step_x, int step_y, Sign target) const;
20     int rank_position(const State& state, int col, int row,
21                      Sign me, Sign enemy) const;
22 };
23
24 }
```

А.2 my_player.cpp

```
1 #include "my_player.hpp"
2
3 namespace ttt::my_player {
4
5 MyPlayer::MyPlayer(const char* id) : player_id(id) {}
6
7 void MyPlayer::set_sign(Sign sign) {
8     player_sign = sign;
9 }
10
11 const char* MyPlayer::get_name() const {
12     return player_id;
13 }
14
15 int MyPlayer::scan_direction(const State& state, int col, int row,
16                             int step_x, int step_y, Sign target) const {
17     int counter = 0;
18     int needed = state.get_opts().win_len;
19
20     for (int step = 1; step < needed; ++step) {
21         int new_col = col + step_x * step;
22         int new_row = row + step_y * step;
23
24         if (new_col < 0 || new_row < 0 ||
25             new_col >= state.get_opts().cols ||
26             new_row >= state.get_opts().rows) {
27             break;
28         }
29
30         if (state.get_value(new_col, new_row) == target) {
31             ++counter;
32         } else {
33             break;
34         }
35     }
36 }
```

```

36     return counter;
37 }
38
39 int MyPlayer::rank_position(const State& state, int col, int row,
40                             Sign me, Sign enemy) const {
41     int total_score = 0;
42     int win_len = state.get_opts().win_len;
43     const int dirs[4][2] = {{1, 0}, {0, 1}, {1, 1}, {1, -1}};
44
45     for (int d = 0; d < 4; ++d) {
46         int dx = dirs[d][0];
47         int dy = dirs[d][1];
48
49         int my_streak = 1 + scan_direction(state, col, row, dx, dy, me)
50             + scan_direction(state, col, row, -dx, -dy, me);
51
52         int enemy_streak = 1 + scan_direction(state, col, row, dx, dy, enemy)
53             + scan_direction(state, col, row, -dx, -dy, enemy);
54
55         if (my_streak >= win_len) total_score += 10000000;
56         if (enemy_streak >= win_len) total_score += 10000000;
57         if (enemy_streak == win_len - 1) total_score += 5000000;
58         if (enemy_streak == win_len - 2) total_score += 500000;
59         if (my_streak == win_len - 1) total_score += 500000;
60         if (my_streak == win_len - 2) total_score += 50000;
61
62         total_score += enemy_streak * 500;
63         total_score += my_streak * 20;
64     }
65     return total_score;
66 }
67
68 Point MyPlayer::make_move(const State& game_state) {
69     int best_value = -1;
70     Point chosen = {0, 0};
71     int width = game_state.get_opts().cols;
72     int height = game_state.get_opts().rows;
73     Sign opponent = (player_sign == Sign::X) ? Sign::O : Sign::X;
74
75     for (int row = 0; row < height; ++row) {
76         for (int col = 0; col < width; ++col) {
77             if (game_state.get_value(col, row) != Sign::NONE) {
78                 continue;
79             }
80             int current_rank = rank_position(game_state, col, row,
81                                             player_sign, opponent);
82             if (current_rank > best_value) {
83                 best_value = current_rank;
84                 chosen = {col, row};
85             }
86         }
87     }
88     return chosen;
89 }
90
91 }

```